Experiment No. – 6						
Date of Performance:	**16/3/2026**					
Date of Submission:	**23/3/2026**					
Conceptual Understanding (Cognitive) (20%) (02)	Code Execution (Psychomotor) (15%) (1.5)	Problem-Solving & Debugging (Cognitive + Psychomotor) (20%) (02)	Professional Conduct & Lab Ethics (Affective) (20%) (02)	Output / Result Accuracy (Product) (15%) (1.5)	Documentation & Result Analysis (Cognitive) (10%) (01)	Sign with Date

Experiment No. 6

1.1 Aim: Simulate a basic blockchain in Python (simulate blocks, hash chaining, and proof-of-work)

1.2 Course Outcome: Apply Blockchain and Distributed Ledger Technology concept to solve real world problems

1.3 Learning Objectives: Simulate a basic blockchain

1.4 Requirement:

Hardware Requirements

- Computer/Laptop
- Minimum 4 GB RAM
- Processor: Intel i3 or higher
- Hard Disk: 500 GB or above

Software Requirements

- Python 3.x
  Python is used to implement the blockchain simulation.
- Python IDE / Code Editor
  - IDLE (Python default editor)
  - Visual Studio Code
  - PyCharm
    (Any one of these can be used)

Python Libraries Used
The following built-in libraries are used in the program:

1. hashlib
   - Used to generate SHA-256 hash values for blocks.
2. time
   - Used to add timestamps to each block.

Programming Language
- Python

Operating System
- Windows 10 / Windows 11
- Linux / macOS (optional)

1.5 Related Theory:

Blockchain is a decentralized and distributed digital ledger used to record transactions securely across multiple computers. Each record in a blockchain is stored in a block, and these blocks are connected together using cryptographic hashes, forming a continuous chain known as a blockchain. Because of this structure, once data is added to the blockchain, it becomes very difficult to modify or tamper with it.
A block is the fundamental unit of a blockchain. Each block typically contains information such as the block index, timestamp, transaction data, previous block hash, nonce value, and the current block hash. The inclusion of the previous block's hash in every block creates a link between blocks, which ensures the integrity and security of the entire chain.
Hashing is an important concept used in blockchain technology. A hash function converts input data into a fixed-length string of characters. In most blockchain systems, the SHA-256 algorithm is used to generate a secure hash value. Even a small change in the block data produces a completely different hash value, making the blockchain resistant to tampering.
Hash chaining refers to the process where each block stores the hash of the previous block. This creates a secure link between blocks. If any data in a block is altered, its hash will change, which will break the chain and indicate that the data has been modified.
Another important mechanism used in blockchain is Proof of Work. Proof-of-Work is a consensus algorithm used to validate and add new blocks to the blockchain. In this process, a special value called a nonce is repeatedly changed until a hash satisfying a certain condition (such as starting with a specific number of zeros) is found. This process is known as mining and requires computational effort.
In this experiment, a basic blockchain system is simulated using the Python programming language. The simulation demonstrates how blocks are created, how hashes are generated using cryptographic functions, and how proof-of-work is used to mine blocks. This helps in understanding the fundamental working principles of blockchain technology.

1.6 Procedure:

1. Install Python (version 3.x) on the computer system and verify the installation using the command python --version.

2. Open a Python development environment such as Visual Studio Code, PyCharm, or Python IDLE.

3. Create a new folder for the experiment and inside it create a Python file named blockchain.py.

4. Import the required Python libraries such as hashlib for generating cryptographic hashes and time for recording the timestamp of each block.

5. Define a Block class that contains attributes such as block index, timestamp, data, previous hash, nonce value, and the current hash of the block.

6. Implement a hash generation function using the SHA-256 algorithm to calculate the hash value of each block based on its contents.

7. Implement the Proof of Work method where the program repeatedly changes the nonce value until a hash with a specified number of leading zeros is generated.

8. Create a Blockchain class that manages the list of blocks and includes functions to create the genesis block, get the latest block, and add new blocks to the chain.

9. Initialize the blockchain and add multiple blocks containing sample transaction data.

10. Execute the Python program to simulate the mining process and generate valid hashes for each block.

11. Display the blockchain data including block index, timestamp, transaction data, previous hash, current hash, and nonce value.

12. Verify that each block correctly stores the hash of the previous block, confirming the hash chaining mechanism.

1.7 Program and Output:

1. Create a new folder on your system
   Example:

Blockchain_Experiment

2. Inside the folder create a new file:

blockchain.py

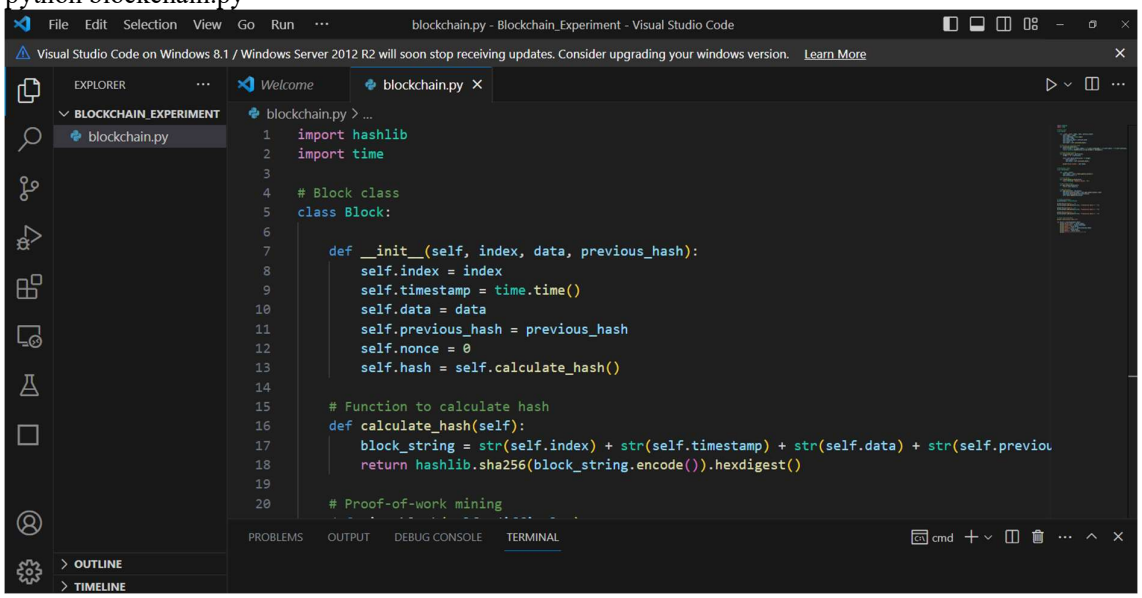
```
import hashlib
import time

# Block class
class Block:

    def __init__(self, index, data, previous_hash):
```

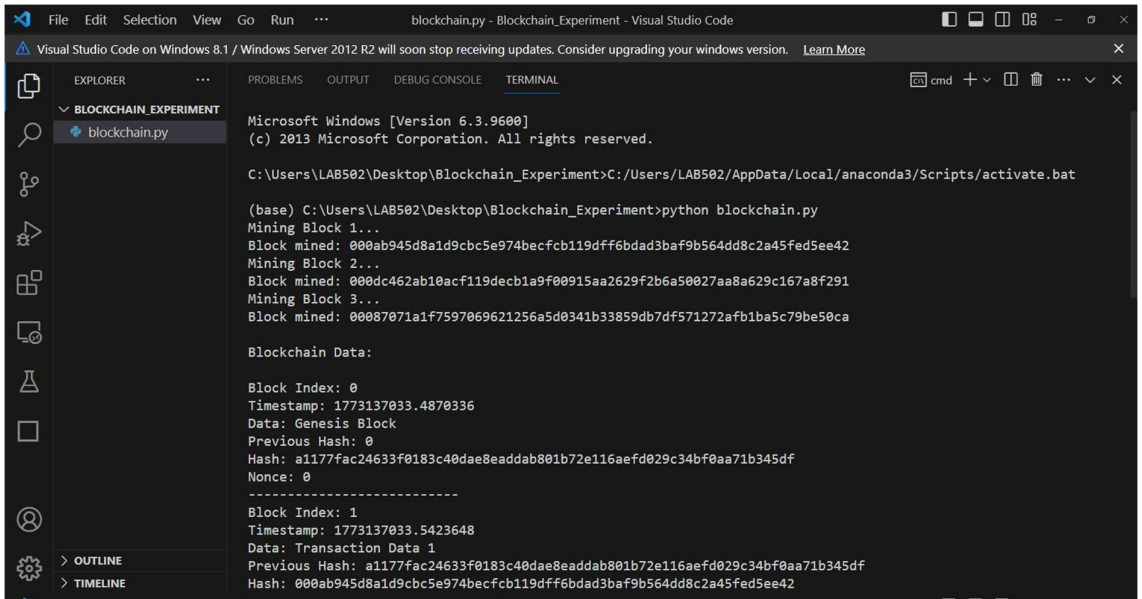
```
        self.index = index
        self.timestamp = time.time()
        self.data = data
        self.previous_hash = previous_hash
        self.nonce = 0
        self.hash = self.calculate_hash()

    # Function to calculate hash
    def calculate_hash(self):
        block_string = str(self.index) + str(self.timestamp) + str(self.data) +
str(self.previous_hash) + str(self.nonce)
        return hashlib.sha256(block_string.encode()).hexdigest()

    # Proof-of-work mining
    def mine_block(self, difficulty):
        target = "0" * difficulty

        while self.hash[:difficulty] != target:
            self.nonce += 1
            self.hash = self.calculate_hash()

        print("Block mined:", self.hash)


# Blockchain class
class Blockchain:

    def __init__(self):
        self.chain = [self.create_genesis_block()]
        self.difficulty = 3

    # First block
    def create_genesis_block(self):
        return Block(0, "Genesis Block", "0")

    # Get latest block
    def get_latest_block(self):
        return self.chain[-1]

    # Add new block
    def add_block(self, new_block):
        new_block.previous_hash = self.get_latest_block().hash
        new_block.mine_block(self.difficulty)
        self.chain.append(new_block)


# Create blockchain
my_blockchain = Blockchain()
```

```
print("Mining Block 1...")
my_blockchain.add_block(Block(1, "Transaction Data 1", ""))

print("Mining Block 2...")
my_blockchain.add_block(Block(2, "Transaction Data 2", ""))

print("Mining Block 3...")
my_blockchain.add_block(Block(3, "Transaction Data 3", ""))


# Print the blockchain
print("\nBlockchain Data:\n")

for block in my_blockchain.chain:
    print("Block Index:", block.index)
    print("Timestamp:", block.timestamp)
    print("Data:", block.data)
    print("Previous Hash:", block.previous_hash)
    print("Hash:", block.hash)
    print("Nonce:", block.nonce)
    print("----------------------------")
```

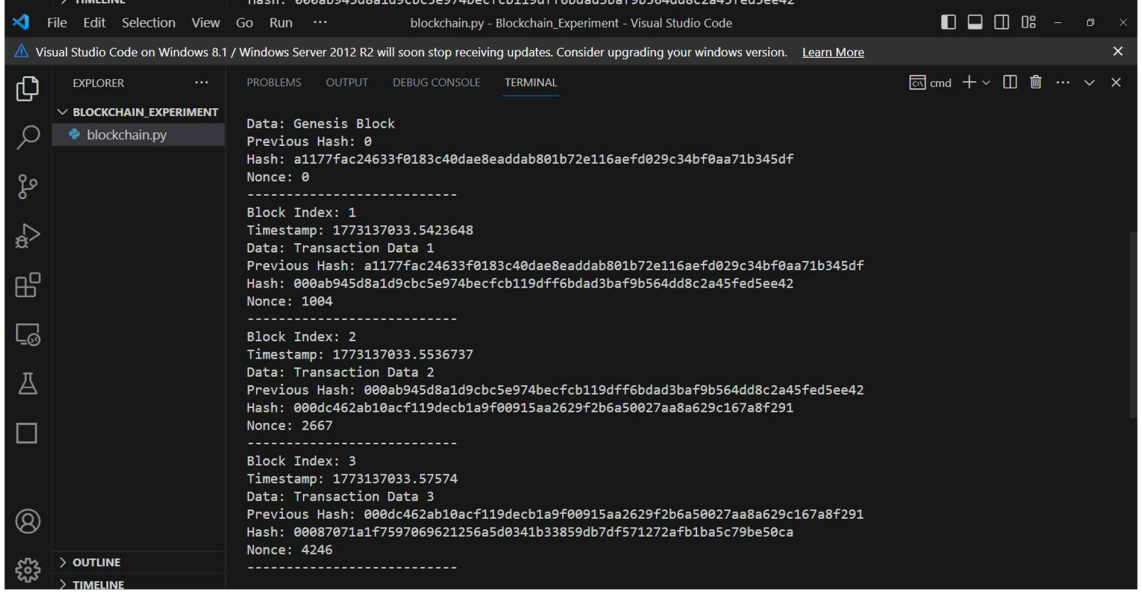
Run the Program

Open terminal in the folder and run:
python blockchain.py





```
Microsoft Windows [Version 6.3.9600]
(c) 2013 Microsoft Corporation. All rights reserved.

C:\Users\LAB502\Desktop\Blockchain_Experiment>C:/Users/LAB502/AppData/Local/anaconda3/Scripts/activate.bat

(base) C:\Users\LAB502\Desktop\Blockchain_Experiment>python blockchain.py
Mining Block 1...
Block mined: 000ab945d8a1d9cbc5e974becfcb119dff6bdad3baf9b564dd8c2a45fed5ee42
Mining Block 2...
Block mined: 000dc462ab10acf119decb1a9f00915aa2629f2b6a50027aa8a629c167a8f291
Mining Block 3...
Block mined: 00087071a1f7597069621256a5d0341b33859db7df571272afb1ba5c79be50ca

Blockchain Data:

Block Index: 0
Timestamp: 1773137033.4870336
Data: Genesis Block
Previous Hash: 0
Hash: a1177fac24633f0183c40dae8eaddab801b72e116aefd029c34bf0aa71b345df
Nonce: 0
--------------------------
Block Index: 1
Timestamp: 1773137033.5423648
Data: Transaction Data 1
Previous Hash: a1177fac24633f0183c40dae8eaddab801b72e116aefd029c34bf0aa71b345df
Hash: 000ab945d8a1d9cbc5e974becfcb119dff6bdad3baf9b564dd8c2a45fed5ee42
```



```
Data: Genesis Block
Previous Hash: 0
Hash: a1177fac24633f0183c40dae8eaddab801b72e116aefd029c34bf0aa71b345df
Nonce: 0
--------------------------
Block Index: 1
Timestamp: 1773137033.5423648
Data: Transaction Data 1
Previous Hash: a1177fac24633f0183c40dae8eaddab801b72e116aefd029c34bf0aa71b345df
Hash: 000ab945d8a1d9cbc5e974becfcb119dff6bdad3baf9b564dd8c2a45fed5ee42
Nonce: 1004
--------------------------
Block Index: 2
Timestamp: 1773137033.5536737
Data: Transaction Data 2
Previous Hash: 000ab945d8a1d9cbc5e974becfcb119dff6bdad3baf9b564dd8c2a45fed5ee42
Hash: 000dc462ab10acf119decb1a9f00915aa2629f2b6a50027aa8a629c167a8f291
Nonce: 2667
--------------------------
Block Index: 3
Timestamp: 1773137033.57574
Data: Transaction Data 3
Previous Hash: 000dc462ab10acf119decb1a9f00915aa2629f2b6a50027aa8a629c167a8f291
Hash: 00087071a1f7597069621256a5d0341b33859db7df571272afb1ba5c79be50ca
Nonce: 4246
--------------------------
```

1.8 Conclusion: In this Experiment we have learned about web3 also designed and deployed smart contract using Metamask and web3.